

del.py 交易逻辑流程图

根据 del.py（源码编码：GBK）提取；本文档使用 UTF-8 生成。只描述现有逻辑，不改动业务代码。

文件定位

脚本角色： 通达信预警信号到 QMT/xtquant 委托执行的自动交易脚本。	主入口： init(C) 初始化，on_timer(C) 每 3 秒轮询。
核心链路： 读预警文件 -> 防重 -> 生成委托 -> 盘口追单 -> 写 response。	关键文件： buy.txt、buy_response_YYYYMMDD.txt、trade_record_log.txt、position_cost.json。

主流程图

读图顺序：先从 A 泳道完成启动，再进入 B 泳道的每轮轮询；B 过滤出新增预警后，进入 C 逐条转委托，最后由 D 执行并写回 response。

A. 启动与定时器	B. 每轮 on_timer
1. init(C) 启动 写“策略启动测试日志”，进入初始化链路。 2. 检查 XML 配置 create_xml_if_not_exists() 不存在则创建，随后清空预警文件并返回。 3. 读取界面参数 set_param() 写入开始/结束时间、最大买入数、价格类型、预警文件、防重规则等。 4. 设置资金账号 set_account() 读取 account 并执行 C.set_account(g.acclD)。 5. 启动时清理 若当天 response 不存在则清空 buy.txt；随后撤掉启动前所有委托。	8. on_timer(C) 进入 读取当前时间，先做 15:35 后的空仓成本清理。 9. 交易时间过滤 早于开始时间或晚于结束时间，直接返回，不读预警。 10. 处理未完成委托 check_unfinished_orders() 扫 ORDER，可能撤单、等待 tick、构造补单。 11. 读取 buy.txt load_tdx_signal() 按 7 列读取：代码、名称、时间、价格、涨跌幅、量、公式。 12. 预警为空则返回 文件不存在、读取异常或 DataFrame 为空时，本轮结束。 13. 识别买卖标记

6. 导出成本快照

`export_official_position_costs()` 查询持仓和历史成交，写官方/历史成本文件。

7. 注册定时器

`C.run_time("on_timer", "3nSecond", ...)` 后循环进入主轮询。

`add_signal_flags()`: 弱卖标记卖出，五日内六级标记买入。

14. 丢弃无关公式

只保留 `is_sell == 1` 或 `is_buy == 1` 的预警行。

15. 加载当天 response

`load_tdx_response()` 读取
`buy_response_YYYYMMDD.txt`，不存在则用空表。

16. response 防重

按配置选择 `stock_code + is_sell` 或 `stock_code + datetime + formula` 作为已处理键。

17. 没有新增预警则返回

防重过滤后为空，本轮结束；否则写“新增预警”日志。

C. 逐条预警转委托

18. 逐行遍历新增预警

每一行都会先转成 `signal_info`，后续追加处理结果。

19. 最大买入标的数检查

买入信号且 `response` 中已成交买入股票数达到上限，则直接生成 `skipped`。

20. convert_order()

设置方向：卖出为 24，买入为 23；补市场后缀。

21. 涨跌幅过滤

30/688 超过 19.5%，60/00 超过 9.5%，设置 `trade_amount = 0` 并跳过。

22. 买入分支

只接受五日内六级；读取总资产和当前持仓市值，目标是补到总资产 2%。

23. 买入金额计算

D. 盘口执行与结果写回

27. do_order(C, order)

若 `trade_amount <= 0`，直接返回 `skipped`。

28. 读取追单参数

默认：撤单等待 1 tick，重新下单等待 2 tick，盘口最大轮数 10，最大滑点比例 0.03。

29. 买入追单

取卖一价量，滑点超限则等待；按金额折算一手整数后提交限价单。

30. 卖出追单

取买一价量，滑点超限则等待；按剩余卖出量提交限价单。

31. 提交、等待、撤单

`submit_limit_order()` 下单，等待 tick，查询成交量，再撤残单。

$buy_amount = total_asset * target_ratio - current_value$; 小于等于 0 则跳过。 24. 卖出分支 查询持仓；没有持仓、可用为 0、成本为空、当前价为空都设置卖出数量为 0。 25. 卖出成本与分档 优先历史成交流水成本，兜底官方成本；150% 卖半，200% 清仓。 26. 生成委托日志 把转换后的 order 写入“生成委托信息”。	32. 维护成交状态 买入成交后更新 position_cost.json；150% 卖半成交后标记 half_sell_done。 33. 生成处理结果 make_process_result() 输出 filled、partial_filled、submitted、skipped 或 failed。 34. 追加到 response 表 add_process_result_to_signal() 加上状态、原因、委托号、成交量、剩余量、处理时间。 35. 保存 response save_tdx_signal_response() 用 GBK 写回制表符文本。 36. 收尾日志 保存后记录账户、持仓、委托、成交，并再次导出官方/历史成本。
--	---

分支逻辑

买入链路 - 信号：formula == 五日内六级 才允许买入。 - 仓位：读取账户总资产，单票目标市值为 总资产 * 单次买入仓位比例，默认 2%。 - 金额：当前持仓市值已达到目标则跳过；否则按差额生成买入金额。 - 执行：盘口追单吃卖一，按卖一价和卖一量折算一手整数，成交后维护 position_cost.json。	卖出链路 - 持仓：先确认股票在持仓池且可用数量大于 0。 - 成本：优先使用历史成交流水成本，失败时用官方持仓成本兜底。 - 分档：当前价达到成本 150% 时卖出 50%，达到 200% 时清仓。 - 执行：盘口追单吃买一，按买一量和剩余卖出量循环提交限价单。
未完成委托处理 - 扫描：check_unfinished_orders() 查询 ORDER，筛选未完成委托。 - 买入：已有部分成交则回填自维护成本；若目标仓位已达成，撤回股票未完成买单。	文件和日志 - 输入：buy.txt 为通达信预警输入，列包含代码、名称、时间、价格、涨跌幅、量、公式。 - 输出：buy_response_YYYYMMDD.txt 记录每条信号处理状态。

- 补单：撤原委托，等待 tick，再按剩余数量构造重试委托。 - 防重：已处理的委托号进入 g.retried_order_ids，避免重复补单。	- 日志：trade_record_log.txt 记录参数、预警、委托、成交、持仓和异常。 - 成本：position_cost.json 存买入累计金额、累计股数、成本价和 150% 卖半标记。
--	---

追单参数细节

参数	默认值	代码使用位置	含义
撤单等待 tick 数	1	do_order() 每次提交限价单后	下单后等待 1 个盘口 tick，再查询成交量并撤掉剩余未成交委托。
重新下单等待 tick 数	2	do_order() 盘口为空、滑点超限、本轮未全部成交后； check_unfinished_orders() 撤原单后	不立即重试，等待 2 个盘口 tick，让盘口刷新后再取新对手价。
盘口最大轮数	10	do_order() 买入追单、卖出追单、未完成买入补单循环	最多尝试 10 轮盘口追单；每轮可能提交一次限价单，也可能因为盘口为空或滑点超限只等待不下单。
最大滑点比例	0.03	is_slippage_exceeded(signal_price, quote_price, max_slippage_ratio)	对手价相对预警价偏离超过 3% 时，本轮不下单，等待重新下单 tick 后再看盘口。

关键函数索引

函数	作用	主要调用
init(C)	初始化配置、账户、response 清理、启动撤单、导出成本，并注册定时器。	create_xml_if_not_exists, set_param, set_account, cancel_all_orders_on_startup
on_timer(C)	交易时间内读取预警，过滤重复和无效信号，逐条生成委托并写 response。	check_unfinished_orders, load_tdx_signal, convert_order,

		do_order
convert_order(order_info)	把预警行转换成交易委托，包含涨跌幅过滤、买入仓位计算、卖出分档计算。	add_market_suffix, get_total_asset, get_history_position_cost_price
do_order(C, order)	按盘口对手价追单，控制滑点，循环提交限价单、等待成交并撤残单。	get_best_opposite_quote, submit_limit_order, get_order_traded_volume, cancel_order_if_possible
check_unfinished_orders(C)	处理历史未完成委托，撤单后按剩余数量补单，并维护买入成本。	is_unfinished_order, build_retry_order, do_order
save_tdx_signal_response(df, path)	标准化 response 列，GBK 写回文本，并导出交易/持仓/成本快照。	normalize_response_columns, log_all_trade_records, log_current_positions, export_official_position_costs

注意点

潜在变量问题：convert_order() 的 150% 卖出分支使用了 self_cost_info.get('half_sell_done')，但在当前函数片段内没有看到 self_cost_info 的赋值。这里没有修改业务逻辑，只标注为需要人工复核的点。